

Two Erasures and Two Laxities

Internalising the Cost and History Monads over Continued Interactive GSLTs

L. G. Meredith*
F1R3FLY.io

July 2026 — working note

Abstract

Two constructions over the category ciGSLT of continued interactive graph-structured lambda theories install an apparatus and then run with it. The cost endofunctor C meters a theory by gating each step on the consumption of a token; the history endofunctor H logs each step as an event, recovering reversibility. Both are monads, and both come with an evident free-forgetful resolution whose forgetful leg strips the apparatus off. This note is about a *second* erasure, available for both and of an entirely different character: rather than stripping the apparatus, one encodes it back into the base theory, so that the metering or the logging is performed by the base’s own computation. We argue that this second erasure is not an adjunction and should not be made into one. Both erasures are algebra structures for the monad on the base object — forgetting is the trivial action, internalising the faithful one — so the pair sits inside a lattice of graded lax algebras ordered by the kernel of the action, and the two resolutions the cost note reached for are simply the Kleisli and Eilenberg–Moore resolutions of one monad. This also corrects the hypothesis under which internalisation was previously claimed: Turing completeness supplies first-order data, whereas internalising a theory’s own cost or history apparatus is a demand for the theory to code its own higher-order syntax, and the two routes to that demand are erasure by binding and reflection. The failure of an internalising algebra to be strict then splits into two kinds. *Graded* laxity is overhead: the algebra laws hold up to relative bisimulation, at a cost measured in the monad’s own monoid. *Kernelled* laxity is loss: the algebra map identifies configurations the source distinguishes, and factors through a quotient. We run the analysis over three probes — CCS+rec , λ , and ρ — and find that they do not differ over whether internalisation exists but over which laxity it suffers, with the grade decomposing into a positional and a representational factor on which λ and ρ sit at opposite corners. The duality that motivates the programme — cost as resource-bounded forward evolution, history as the record required for reversal — is then not a single theorem but a hierarchy of internality across the three probes.

Contents

1	The programme	2
2	The setting	3
2.1	Rung one: graph-structured	3
2.2	Rung two: interactive	3
2.3	Rung three: continued	4
2.4	Two carriers of the surface	4

*lgreg.meredith@f1r3fly.io

3	Relative expressiveness	5
3.1	Contexts as the primary structure	5
3.2	Internal representability, and a clarification	5
4	Two monads	7
4.1	The cost monad	7
4.2	The history monad	7
5	Two erasures	7
5.1	Forgetting	7
5.2	Internalising	7
5.3	Both erasures are algebras	8
6	The gadget: a lattice of graded lax algebras	9
6.1	Grading in the monad's own monoid	9
6.2	When is there a monad map?	9
7	Two kinds of laxity	10
8	The three probes	11
8.1	CCS+rec: no plug, absolutely or relatively	11
8.2	λ : sharp positions, expensive codes	12
8.3	ρ : cheap codes, collapsed positions	13
8.4	The profile	13
9	The duality, as a hierarchy of internality	13
10	Open problems	14

1 The programme

Three prior notes set up the pieces. The cost endofunctor note [1] isolates the class of theories on which cost accounting can be installed — continued interactive GSLTs, whose dynamics is presented as an interaction cut and which carry a computable section of their structural-congruence quotient — and builds an endofunctor C that signs terms, adjoins a token stack, and replaces the cut by a gated family, so that no redex fires without spending a token. The probing-universality note [2] replaces the Turing yardstick with a relative property: one fixes a probe theory and asks what that probe can see, with contexts as the primary structure and two conditions — hosting and exhausting — preventing the question from having a trivial answer. The history note [3] adds the complementary construction H , the writer monad over rewrite events, whose free instance unfolds the reduction graph into its universal cover and whose unique-parent property is reversibility.

Each of C and H arrives with a free-forgetful adjunction. *Install* the apparatus, *forget* the apparatus. Forgetting is structure-preserving and behaviour-altering: strip the gating and the theory runs without friction; strip the log and it runs without a past.

There is a second way to be rid of the apparatus, and it is the subject of this note. Instead of deleting it, encode it. If the base theory can represent the tokens, the stacks, the signatures — or the events, the occurrences, the residuals — as data of its own, then the metered or logged dynamics can be simulated by the base's own computation, and the adjoined sorts dissolve without the behaviour they governed being lost. Call this *internalisation*. It is an erasure of the apparatus and not of what the apparatus did.

The question this note sets out to answer is what kind of gadget internalisation is. The cost note guessed at an adjunction [1, §9.3], and we will argue that the guess is wrong in three separable ways: the hypothesis is at the wrong level of the linear-time/branching-time spectrum, it is the wrong property even at the right level, and the conclusion as stated is vacuous. All three defects are repaired by machinery already present in [2], and the repair yields a better object than the one originally sought.

Plan. Section 2 recalls ciGSLT. Section 3 recalls the relative notion of expressiveness and makes one clarification on which everything downstream turns: internal representability is a property of a morphism, not of an object. Section 4 recalls C and H and the sense in which they are two ledgers of one clock. Section 5 sets the two erasures side by side and observes that both are algebras. Section 6 proposes the gadget: a lattice of graded lax algebras. Section 7 separates the two kinds of laxity. Section 8 runs the three probes. Section 9 states the duality as a hierarchy of internality. Section 10 lists what is not settled.

Register. This is a working note. Several statements are given at the level of rigour the setting affords, and the load-bearing gaps are flagged as such rather than papered over.

2 The setting

We recall only what is needed, and we take the rungs in order, because the distinction between the second and the third is easy to collapse and is load-bearing throughout what follows.

2.1 Rung one: graph-structured

A graph-structured lambda theory is a multi-sorted term theory $G = (\Sigma, E, R)$: a sorted signature with binding, a set of terms generated by it, a structural congruence $\equiv$ making the term set a graph-like quotient, and a rewrite relation $\rightarrow$ on the quotient. Almost everything qualifies, including Turing machines with $\equiv$ trivial, and that breadth is the problem: the shape of the dynamics is not constrained at all, so nothing can be said uniformly about how one theory simulates another.

2.2 Rung two: interactive

Definition 2.1 (Interactive). G is *interactive* when it carries, over and above the data of rung one, a distinguished interacting sort P of programs; a binary interaction constructor $K(P, E)$ representing the interaction of a program with an environment E , an operand of the interacting sort; and at least one base rewrite rule whose left-hand side features K [2, Def. 2.2].

That is the whole of the rung, and it is worth displaying what such a rule looks like, since it is considerably less than what the rest of this note will use. A merely interactive theory has a generating rule of the shape

$$K(A, B) \longrightarrow r \quad (\ddagger)$$

with A and B patterns over the interacting sort and r a term over their variables. The rule is K -headed, and that is the entire assertion. There is no *surface*, because no part of A has been designated as the part that must match a part of B ; no *continuation*, because nothing has been designated as the rest; and no *contraction*, because r is given outright rather than

computed from designated pieces of A and B . Interactive names a site of interaction and a rule that fires there. It does not decompose either operand of the cut, and in particular it supplies neither of the two things — surfaces and continuations — on which Sections 8 and 9 turn.

2.3 Rung three: continued

The third rung decomposes both sides. It is here, and not before, that surfaces and continuations appear.

Definition 2.2 (The interaction cut). An interactive GSLT is presented in *interaction-cut form* when its generating rule can be written

$$K(K_p(I, C_p), K_e(J, C_e)) \longrightarrow K'(C'_p, C'_e), \quad (C'_p, C'_e) = \text{compute}(I, J, C_p, C_e), \quad (\dagger)$$

naming the interaction constructor K ; two introductions $K_p(I, C_p)$ and $K_e(J, C_e)$, each separating an interaction surface (I, J — the part that must match for the cut to fire) from a continuation (C_p the program continuation, C_e the environment continuation); the partial contraction compute , Milner’s pseudo-application, which on a surface match combines the continuations into residuals; and the constructor K' repackaging them.

Comparing $(\dagger)$ with $(\ddagger)$: the operands A and B have been required to be introductions, each factoring into a surface and a continuation, and the right-hand side has been required to be determined by a partial operation on the four resulting pieces. Neither requirement is available at rung two.

Binding is one mode by which information migrates across the contraction, not a feature of the cut: CCS realises the shape with pseudo-application degenerating to pure release, and is in that sense the paradigmatic instance.

Definition 2.3 (Continued). An interactive GSLT is *continued* when equipped with a presentation in the form $(\dagger)$ and with a *section*: a computable canonical-form function on the interacting sort. ciGSLT is the category of these.

Remark 2.4 (Where each rung bites). The gap between rungs one and two is the one that excludes the machine models: a Turing machine’s natural cut is between automaton and tape, which inhabit different sorts, so no binary K on the interacting sort has the transition relation as an instance, and the same holds of Mealy and Moore machines [2, Rmk. 2.3].

The gap between rungs two and three is thinner among the usual suspects — CCS, π , ρ , λ , the ambient calculus, and the interaction categories are all continued — and is populated by theories whose K -headed rule does not factor. A rule firing on a global condition, say $K(P, Q) \rightarrow 0$ whenever $P \equiv Q$, is of the shape $(\ddagger)$ and admits no reading as $(\dagger)$: the match condition is the whole of both operands, so there is no part that is the surface and no remainder that is the continuation. Such a theory is interactive and not continued. The rung is therefore doing definitional work rather than excluding a favourite calculus, which is exactly why it is easy to skip over — and skipping it is how the cost note came to attribute to interactive theories a structure only continued ones have.

2.4 Two carriers of the surface

Two facts about surfaces will matter, and both are rung-three facts. When K is free, position is unforgeable and the surface is carried structurally, by the context $K([], S)$; when K carries any equation, position is forgeable and the surface must be reified as an explicit datum, matched by a partial nearness operator $\text{near}(I, J)$ [2, Rmk. 2.7]. The two poles are a free K (λ , where $K = \text{App}$) and a freely mixed associative–commutative K (ρ and CCS, where $K = |$).

3 Relative expressiveness

3.1 Contexts as the primary structure

The move of [2] is to take contexts rather than terms as primary. A theory presents a symmetric multicategory $\text{Ctx}(G)$ whose objects are interfaces (s, n, σ) — sort, binding stage, surface multiset — and whose morphisms are multi-hole contexts taken modulo $\equiv$. Terms are the nullary contexts, so a functor $\text{Ctx}(G) \rightarrow \text{Ctx}(H)$ encodes terms and contexts by one and the same map. Transitions are labelled by contexts: $P \xrightarrow{C} Q$ iff $C \in L$ and $C[P] \rightarrow Q$, with the only primitive observation being $\Downarrow$, a reduction or a success report. No modal logic is used.

A morphism $\iota : G \rightarrow H$ is a symmetric pseudofunctor of context multicategories, stage-indexed, preserving and reflecting $\Downarrow$. It induces a label class $L_\iota = \iota(\text{Ctx}_1(G))$, and the target is observed *only* through that class: $\sim_\iota$ is bisimulation in $\text{CLTS}(H, L_\iota)$. This is what allows the extra expressive power of a target to be recorded as data about an encoding rather than as an obstruction to its existence.

Remark 3.1 (Which rung this needs). The surface component σ of an interface is the one piece of the above that is not available at rung two, since surfaces are introduced only with the cut ($\dagger$). It is a refinement rather than a prerequisite: the construction runs at rung two throughout, and dropping σ costs resolution in the labels and nothing else [2, Rmk. 2.8]. What genuinely requires rung three is the *section*, which is what makes $\text{Ctx}(G)$ effectively presented — contexts admit codes, equality is decidable on representatives, hole-filling is computable [2, Prop. 2.9] — and hence what makes the interpreters of Section 8 exist as terms rather than as relations between terms. Every internalisation claim below depends on that clause and not on the surface component.

Definition 3.2 (The two conditions). ι *hosts* G , written $\iota \models (\text{F})$, if it is faithful up to bisimulation: $\iota(C) \sim_\iota \iota(D)$ implies $C \sim_{\text{Ctx}_1(G)} D$. Its *kernel* is the congruence $C \approx_\iota D \iff \iota(C) \sim_\iota \iota(D)$; hosting says the kernel is as fine as possible. ι *exhausts* H , written $\iota \models (\text{D})$, if it is dense: every H -context is $\sim_\iota$ -indistinguishable from an assembly of ι -images by parallel composition, restriction, and hole-filling.

Hosting says the target can run the source; exhausting says the target is nothing the source cannot assemble. Together they say the two can write meta-circular interpreters for one another [2, Cor. 6.4]. Every morphism factors as a quotient by its kernel followed by a hosting embedding [2, Prop. 4.11]; we will use this factorisation as the diagnostic that separates our two laxities.

3.2 Internal representability, and a clarification

Definition 3.3 (Condition (I)). $\iota : G \rightarrow H$ satisfies (I) if there is a coding $\ulcorner - \urcorner$ of G -terms as H -data, definable in H , and a single H -term U_ι with $U_\iota \langle \ulcorner P \urcorner \rangle \sim_\iota \iota(P)$ for every G -term P .

Condition (I) is what turns a translation into an interpreter, and it is the condition on which internalisation rests. One clarification is essential, and its absence is the source of the error we are repairing.

Remark 3.4 ((I) is a property of a morphism, not of an object). The literature invites the reading “theory T satisfies (I),” and [2] itself uses that shorthand in two places that appear to conflict. Its separations table records $\text{TM}^\sharp \rightarrow \text{CCS} + \text{rec}$ as satisfying (I); its Remark 2.13 records that λ , π , and ρ satisfy (I) and that CCS does not. Both are correct, because (I) is indexed by a source, and internalisation is always the endo-directed instance: its source is

the theory's own apparatus, so the coding required is *self*-coding. Nothing about a theory's power to code *other* models transfers, and Section 8 turns on the difference.

The clause of Definition 3.3 that does the work is not the existence of $\ulcorner - \urcorner$ and U_i but the requirement that $U_i(\ulcorner P \urcorner) \sim_i \iota(P)$. Codings are cheap; correct ones are not. It is worth separating the two ways a theory can come by one.

Definition 3.5 (Absolute, relative, and Gödel self-coding). A self-coding of G is *absolutely native* when hole-filling is realised by an operation G already has, in a single step and with no decoding: a one-hole context is representable as a term $C[] \mapsto \lambda x.C[x]$ with plug realised by substitution — *erasure by binding*, available in λ, π, ρ — or as a name, with plug realised by dereference — *reflection*, available in ρ , and there twice over, since $@$ makes the representing term quotable in turn.

It is *relatively native* when there is a theory N with an absolutely native plug and a pair of Ctx-morphisms $j : G \rightarrow N$ and $k : N \rightarrow G$, both satisfying (F), such that plugging is realised as k applied to a single N -step. The plug need not be an operation G itself has, only the image of a one-step operation in a theory G faithfully embeds into and which faithfully embeds back.

It is *Gödel* when the code is a numeral or comparable first-order datum and plug is realised by an interpreter that must decode it.

Remark 3.6 (Gödel self-coding is always available). Any Turing-complete theory Gödel-codes its own contexts: its syntax is countable, and a universal process supplies the interpreter. One therefore cannot argue that a theory fails (I) on the grounds that no coding exists, and the following composite establishes exactly that much. CCS+rec is Turing complete, so it codes λ ; λ self-codes natively; so CCS+rec codes λ 's contexts, and by Gödel-coding $\text{Ctx}(\text{CCS+rec})$ into λ first, its own. The distinction of Definition 3.5 is thus not about what can be coded but about what plugging costs.

Remark 3.7 (The ρ route, and where it turns). Relative nativeness is not idle, and the strongest attempt to give CCS+rec a native plug runs through ρ rather than λ . The chain is: ρ codes CCS, faithfully, since ρ subsumes CCS behaviourally and loses no branching on the way in; ρ 's for-comprehension makes a hole a name to receive at, so $\text{Ctx}(\text{CCS}) \rightarrow \rho$ is native — plugging is one communication step, not a decode; and if CCS then coded ρ back at (F)-fidelity, the composite would be a relatively native self-coding of CCS, with no Gödel encoding anywhere and no loss of branching. Every step but the last is sound, and the λ variant of the same chain fails only at the branching that ρ preserves.

The last step is where it turns, and it fails for a reason internal to the programme rather than by appeal to computability. By [2, Cor. 5.10] no Ctx-morphism $\pi \rightarrow \text{CCS+rec}$ satisfies (F), and $\rho \supseteq \pi$, so a fortiori none from ρ . The obstruction is at the level of stages, not of terms: $\mathcal{F}_\pi$ contains non-injective maps, since a π context may deliver a name identifying two previously distinct names and so turn an inert composition into a redex, whereas every map of $\mathcal{F}_{\text{CCS}} = \mathbb{I}$ is injective. The stage functor φ_i would have to send a non-monic to a monic, hence identify maps that $\mathcal{F}_\pi$ distinguishes, contradicting (F) by [2, Prop. 5.9]. That the corollary states CCS+rec's Turing completeness in the same breath is the point: the return leg fails on binding signatures, and computational power is not what is at issue.

A reductio confirms the placement. Were the return leg available at (F)-fidelity we would have $\text{CCS} \equiv \rho$, and with the λ variant λ would join them, so the branching-time order would have a single degree above the Turing threshold — the collapse the relative programme exists to deny, and one that would make [2, Problem 4] unprovable. The last step cannot hold without taking the ambient notion down with it.

4 Two monads

4.1 The cost monad

C adjoins to a continued interactive GSLT a sort **Sig** of signatures with $\# = \text{digest} \circ \text{cf}$ and a commutative monoid $(\text{Sig}, *, ())$; a wrapped-term sort T with $\{\cdot\}_s$; and a sort **Stk** of token stacks $S ::= () \mid s :: S$. Every continuation slot is re-sorted to T , so in the image every continuation is a metered thunk by construction and the contraction need not be lifted. The cut is replaced by a gated family whose only run-time interaction with tokens is unwrapping: forcing a thunk by spending a matching cell [1, §3].

The unit $\eta_G(P) = \{P\}_()$ embeds G as the cost-free fragment; the multiplication μ_G flattens by multiplying nested signatures in $(\text{Sig}, *)$ and concatenating the stack-of-stacks in the free monoid $(\text{Stk}, \cdot)$. It is a monad, and a non-idempotent one: flattening is a true merge of two resource accounts [1, Rmk. 9.3].

4.2 The history monad

H adjoins a history component: configurations are pairs (P, h) with h in the free monoid $M = (\text{Ev}^*, \cdot, \varepsilon)$ on rewrite events, and each rule is logged, $(\ell, h) \rightarrow (r, h \cdot e_{\ell \rightarrow r})$. The unit starts every term with the empty history, the multiplication concatenates, and the augmentation $\pi_G(P, h) = P$ forgets. The free history unfolds the reduction graph into its universal cover, and reversibility is that tree’s unique-parent property [3, §3].

Remark 4.1 (Two ledgers of one clock). The token stack is a forward modulus: produced up front, drained by reduction. The history is the same quantity accrued from the other end: produced by reduction, never drained. Run a cost-accounted term under H from an initial stack of depth σ_0 ; each forced step pops one cell and appends one event, so $\sigma(t) + \kappa(t) = \sigma_0$ throughout [3, Prop. 7.1]. Fuel remaining plus fuel recorded spent is conserved by construction. This is the duality the present note is written to support, and the reason the two monads must be treated by one apparatus.

A detail of Definition 2.1 of [3] carries most of the weight of Section 8: a rewrite event records the *occurrence*, that is the context, at which the rule fired. Events are therefore higher-order data. Internalising H is a demand that the theory code its own contexts; internalising C is a demand that it code its own canonical forms. These are different demands, and ρ satisfies both with one operator only because it fuses section and quotient in @ [1, §5].

5 Two erasures

5.1 Forgetting

For each monad the evident resolution installs and strips. Free $\dashv$ Forget for C , Log $\dashv$ Forget for H . Forgetting is structure-preserving and behaviour-altering: erasing the gating removes the friction, so Forget Free G runs G ’s interactions for nothing, and erasing the log reintroduces irreversibility.

5.2 Internalising

The cost note proposes a second construction [1, §9.3]. Restricting to Turing-complete bases, the metering apparatus of $C(G)$ — signature equality, stack maintenance, gating — is computable, hence encodable in G itself, yielding $I_G : C(G) \rightarrow G$ with $I_G \circ \eta_G \approx \text{id}_G$ up to weak bisimulation and, allegedly, an adjoint retraction $\eta_G \dashv I_G$ in a bicategory of simulations.

Remark 5.1 (Three defects). The construction is sound in spirit and unsound as stated.

Wrong level. Turing completeness is a statement about the input/output relation, the coarsest invariant in the linear-time/branching-time spectrum; the conclusion is stated up to weak bisimulation, at the other end. An interpreter decodes before it acts, decoding is silent work, and silent work relocates choice points, so the inference is exactly the one [2] exists to forbid.

Wrong property. Even granting the level, what internalisation requires is not universality but *faithful* self-coding, and Turing completeness delivers only half of that. It does supply a coding — by Remark 3.6 every Turing-complete theory Gödel-codes its own contexts, so the existence half of (I) is free and no argument should rest on denying it. What it supplies no reason whatever to expect is the behavioural clause $U_\iota(\ulcorner P \urcorner) \sim_\iota \iota(P)$, still less the (F)-strength claim that $I_G \circ \eta_G \approx \text{id}_G$. Turing completeness is silent about fidelity, and fidelity is the whole of what [1, Prop. 9.5] needs.

Vacuous conclusion. If unit and counit are both invertible up to bisimulation then the adjunction is an adjoint equivalence and the word does no work [2, Rmk. 6.6]. The repair there is to grade, so that the failure to be an equivalence is exactly the interpretive overhead [2, Cor. 6.9]; we take up that repair in Section 7.

The signatures of $C(G)$ are digests of canonical forms of G -terms and the events of $H(G)$ carry G -contexts. A Turing-complete base can code both. Whether it can plug them back without paying in observable transitions is a separate question, answered by Definition 3.5 rather than by computability, and it is the question Section 8 decides.

Note the mechanism of the repair, since it is the whole point. The cost note asks weak bisimulation to *absorb* the interpreter’s administrative steps. The relative setting instead restricts the observing contexts to L_ι and then recovers the overhead as a grade. Nothing is swept into τ ; the residue moves from the equivalence to the measure.

5.3 Both erasures are algebras

The following is elementary and, we think, decisive.

Proposition 5.2 (The trivial action is an algebra). *Let T be either C or H and let $\alpha_G : T(G) \rightarrow G$ discard the apparatus — for H , $\alpha(P, h) = P$; for C , strip wrappers and ignore the stack. Then $\alpha \circ \eta = \text{id}$ and $\alpha \circ T\alpha = \alpha \circ \mu$. Hence forgetting is an Eilenberg–Moore algebra structure on G : the trivial one.*

Proof. For H : $\eta(P) = (P, \varepsilon)$ and $\alpha(P, \varepsilon) = P$. Both $\alpha \circ H\alpha$ and $\alpha \circ \mu$ send (P, h_2, h_1) to P , since both discard every history component. The C case is identical, with signature multiplication and stack concatenation both discarded. $\square$

Internalisation, when it exists, is likewise a map $T(G) \rightarrow G$ satisfying the same two laws up to the equivalence in play. So the two erasures are not an adjunction and a not-an-adjunction. They are *two algebra structures on the same object*, differing in the kernel of the action: forgetting has maximal kernel, internalising has — when it hosts — trivial kernel.

Remark 5.3 (What the cost note’s two adjunctions actually are). The cost note promises “two adjunctions with opposite embeddings” [1, §9]. That phrase describes the Kleisli and Eilenberg–Moore resolutions of a monad, initial and terminal among resolutions. Its Cost-ciGSLT is the free side, i.e. Kleisli; internalisation lives on the Eilenberg–Moore side, as a lift along $U^C : \text{ciGSLT}^C \rightarrow \text{ciGSLT}$. The note built the first resolution and then went looking for the second in a place where it had already been. The internalisable objects are exactly those admitting such a lift, and the correct hypothesis cutting them out is (I) for the endo-directed morphism, not Turing completeness.

6 The gadget: a lattice of graded lax algebras

6.1 Grading in the monad's own monoid

The grading of [2, Def. 6.7] is a function $g : \mathbb{N} \rightarrow \mathbb{N}$ bounding the target steps per source step, introduced to rescue a degenerate adjunction. For C and H it need not be imported.

Definition 6.1 (Grading object). The grading object of C is the monoid $(\text{Sig}, *) \times (\text{Stk}, \cdot)$; that of H is $(\text{Ev}^*, \cdot)$. In each case it is the monoid the monad already carries, and grades compose under μ by the monad's own multiplication: signatures multiply and stacks concatenate for C , events concatenate for H .

So these monads *measure their own internalisation, in their own currency*. This is a better situation than the general one, and it is what makes the grading intended semantics rather than a rescue: the token stack was already “an exact operational modulus for the cut elimination, realised by running the reduction” [1, Prop. 4.1], and the history is the same modulus accrued backwards.

Definition 6.2 (Graded lax algebra). A *graded lax T -algebra* on G is a Ctx-morphism $\alpha : T(G) \rightarrow G$ together with 2-cells

$$u : \alpha \circ \eta_G \Rightarrow \text{id}_G, \quad a : \alpha \circ T\alpha \Rightarrow \alpha \circ \mu_G$$

in the graded simulation bicategory, each witnessing an $\sim_\iota$ -equivalence and each carrying a grade in the grading object of T , subject to the evident coherence.

Proposition 6.3 (The lattice). *The graded lax T -algebra structures on a fixed G are pre-ordered by the kernel of the action. Proposition 5.2 exhibits a bottom element — the trivial action, maximal kernel — and an internalisation with (F) , when one exists, is a top: trivial kernel. Intermediate algebras are partial erasures, retaining some of the apparatus as data and discarding the rest.*

Remark 6.4 (Relation to the erasure lattice of the history note). Section 4 of [3] constructs a lattice of erasures for H between the reversible cover and the irreversible base, corresponding by covering-space Galois theory to subgroups of $\pi_1(\mathcal{G}(P))$. That is a lattice of *quotient monads* $H_\sim$, i.e. of congruences on M , whereas Proposition 6.3 is a lattice of *actions* of M on a fixed base. These are two lattices related by a Galois connection — an action factors through $H_\sim$ exactly when it kills $\sim$ — and not the same lattice. The identification should not be made casually. Under it, the order-forgetting (Mazurkiewicz) grade of [3, Rmk. 4.3] is the action that kills exactly the transpositions of concurrently enabled steps.

6.2 When is there a monad map?

The most economical packaging would be a single 2-cell rather than per-object structure.

Definition 6.5 (Uniform internalisation). A *lax monad morphism* $T \Rightarrow \text{Id}$ on a full subcategory $\mathcal{K} \subseteq \text{ciGSLT}$ is a family α_G of graded lax T -algebras, natural in Ctx-morphisms of $\mathcal{K}$, with T restricting to $\mathcal{K}$.

This exists exactly when $\mathcal{K}$ has a uniform faithful top: every object internalises, the choice is natural, and T preserves membership. It is therefore a property of a class of theories and not the general gadget. Section 8 exhibits a class where it fails.

Two closure questions attach to Definition 6.5 and are not discharged here. First, whether T restricts: $C(G)$ adjoins Sig and Stk , which are inert under $\rightarrow$ and bind nothing, and by

[2, Rmk. 3.7] they enlarge the sort set while leaving the stage category untouched — so the expectation is that C preserves membership, but the argument that $C(G)$ remains self-coding when G is has not been written down. Second, whether hosting and exhausting transport: hosting does, by transitivity [2, Prop. 4.7], since $M \preceq G \preceq C(G)$ via η ; exhausting is the live question, and we record what it amounts to.

Remark 6.6 (Closure and internalisability are the same condition). For a probe M and an M -exact G , density of $M \rightarrow C(G)$ asks that every $C(G)$ -context be $\sim_t$ -indistinguishable from an assembly of M -images: that M assemble wrappers, stacks, and the gated family. That is precisely the internalisation claim. So C preserves M -exactness exactly when the apparatus internalises, and closure need not be proved separately.

The witnessing assembly is, moreover, already on record. Section 7 of [1] describes what C was built to replace: “fuel acquisition was an encoded multi-step protocol on auxiliary channels,” with C ’s improvement over it being protocol fusion. That translation presentation is the density witness, and the grade of the internalisation is the fusion speedup read backwards — a quantity the cost note named and then discarded.

7 Two kinds of laxity

An internalising algebra can fail to be strict in two ways, and conflating them is the last of the errors we are repairing.

Definition 7.1 (Graded and kernelled laxity). A graded lax T -algebra α on G has

graded laxity

when $\alpha \models (\text{F})$ — the kernel is trivial — and the 2-cells u, a carry non-unit grade. Nothing is identified; the laws hold, at a price.

kernelled laxity

when $\alpha \not\models (\text{F})$: the action identifies configurations the source distinguishes, and by [2, Prop. 4.11] factors as $T(G) \rightarrow T(G)/\approx_\alpha \rightarrow G$ with the second leg hosting.

Graded laxity is *overhead*. Kernelled laxity is *loss*. The distinction is the over-/under-discrimination dichotomy of [2, Rmk. 2.17] seen from the algebra side: a target whose own contexts see too much is repaired by relativising the label class, whereas an encoding that identifies too much is not repairable at all and must be recorded as a kernel.

Proposition 7.2 (The criterion for uniformity). *A lax monad morphism $T \Rightarrow \text{Id}$ on $\mathcal{K}$ tolerates graded laxity and not kernelled laxity: naturality of α together with (F) at each object is what makes the family a family. A class containing an object whose only internalisation is kernelled admits no such morphism, and its lattice of algebras has a top that is not faithful.*

Remark 7.3 (The thermodynamic reading). The two laxities have distinct standing under [3, Rmk. 7.2]. Graded laxity is a bijective re-encoding: the apparatus is kept, in another representation, so the ledger law $\sigma + \kappa = \sigma_0$ survives and the step is logically reversible, hence Landauer-free. Kernelled laxity destroys information that is not recoverable from the state, and is therefore dissipative. Forgetting, being the maximal kernel, is the maximally dissipative erasure; a faithful internalisation is the free one. This is the sharpest available statement of what distinguishes the two erasures, and it is a physical statement rather than a categorical one.

8 The three probes

We now run the analysis over three probes. Fixing a probe M and considering the objects G admitting a Ctx-morphism $M \rightarrow G$ satisfying (F) and (D) gives the $\equiv$ -class of M ; taking a family of probes rather than one gives a *profile* rather than a degree, which is Problem 7 of [2] with the family fixed at $\{\text{CCS+rec}, \lambda, \rho\}$.

8.1 CCS+rec: no plug, absolutely or relatively

CCS+rec is Turing complete; [2, Cor. 5.10] says so explicitly while proving that no Ctx-morphism $\pi \rightarrow \text{CCS+rec}$ satisfies (F). By Remark 3.6 it therefore Gödel-codes its own contexts, and any claim that it simply cannot code them is false. The question is what plugging costs, and it must be settled at both levels of Definition 3.5.

Absolutely, CCS+rec has no plug. A CCS context is a path made of actions, names, and residual bags; the prefix layer differentiates to an action and the restriction layer to a name, neither of which is a term [2, §2.3]. Restriction hides a name but is never instantiated and prefix binds nothing, so there is no abstraction to form and no substitution to plug with, and erasure by binding is unavailable.

That much is a statement about the layer derivative, and on its own it is *not* sufficient — Definition 3.5 allows the plug to be borrowed. *Relatively*, then: the borrowing requires a faithful return leg from a theory with an absolutely native plug, and by Remark 3.7 there is none. The candidate lender is ρ , whose for-comprehension makes hole-filling a single communication step and which loses no branching on the way in; the leg that fails is the return, on binding signatures, at the level of stage categories. So the verdict rests on the programme’s own separation theorem rather than on a syntactic observation about ∂F , which is as it should be: the syntactic observation is what one notices first, and it would be defeated by the ρ chain if the chain closed.

So the internalisation exists, by the Gödel route and only by it, and the question is its fidelity. Gödel-number the configurations, run a universal process, and present an input datum as a synchronisation sequence on bit_0, bit_1, end . That is functionally correct and not $\sim_\iota$ -faithful. In a value-passing calculus receiving a datum is one transition; serialised it is $|\text{code}|$ transitions, each a branch point. Serial decoding manufactures choice points that $\iota(P)$ does not have — the impedance-matching diagnosis of [2, §2.2] turned on CCS instead of on the machine models. And the choice points are *observed*, because in the endo-directed case the label class is generated by CCS+rec’s own contexts, which are exactly the contexts that can synchronise on bit_0 and bit_1 and so separate a decoding in progress from one not yet begun.

That last step is what makes the failure a failure of (I) and not merely of elegance, and it also resolves the apparent conflict noted in Remark 3.4. The morphism $\text{TM}^\sharp \rightarrow \text{CCS+rec}$ satisfies (I) because its source is *already* serial: a process presentation of a Turing machine turns one machine transition into several cut firings, request, reply, and commit [2, §2.2], so serialising costs nothing relative to it. The endo-directed morphism does not, because CCS+rec’s own synchronisation is atomic and serialising it is a strict refinement. Same target, same coding technique, opposite verdicts, decided entirely by the source.

Proposition 8.1 (Two conditions for kernelled laxity). *An internalisation suffers kernelled rather than graded laxity only if both of the following hold: the theory has no plug that is native, absolutely or relatively, so decoding is forced; and the theory is non-confluent, so that the forced decoding steps can relocate choice points. Either alone is insufficient. A confluent theory has no choice points for administrative steps to relocate, so its Gödel coding costs*

grade and never kernel; a non-confluent theory with a native plug never decodes, so nothing is exposed.

CCS+rec satisfies both, which is why it is the interesting column. λ fails the second and ρ fails the first, by different routes, and this is the substance of the profile below rather than a coincidence of two calculi.

Remark 8.2 (Confirmation from the reversibility literature). If H has no faithful internalisation over CCS, then making CCS reversible should require external bookkeeping. It does: CCSK [4] and RCCS [5] obtain backward determinism by adjoining apparatus, never by encoding it into the calculus. The free H exists there; its faithful internalisation does not.

Remark 8.3 (An undischarged hypothesis, and exactly what it would reopen). The CCS+rec column now rests on [2, Cor. 5.10], and that corollary rests in turn on a gap which [2, Rmk. 5.11] flags rather than closes: its argument rules out encodings that are stage-indexed in the sense of [2, Def. 3.14(iii)], but not one that pre-allocates a fixed finite pool of CCS channels and simulates name identification by case analysis over the pool. Excluding those needs a finitary uniformity hypothesis, essentially finite support in the sense of nominal sets.

What hangs on it is precise. If such an encoding existed, the return leg $\rho \rightarrow \text{CCS+rec}$ of Remark 3.7 would be available, CCS+rec would acquire a relatively native plug, the first clause of Proposition 8.1 would fail, and the CCS+rec column would change verdict from kernelled to graded. The reductio of Remark 3.7 says the cost of that would be borne by the ambient notion rather than by this note — the branching-time order would collapse to a single degree above the Turing threshold — but until the hypothesis is discharged the column is conditional, and conditional on this one step and no other.

8.2 λ : sharp positions, expensive codes

λ satisfies (I) endo-directedly by erasure by binding, and carries a section — the de Bruijn form [1, §5.2]. Both erasures therefore exist, and the internalisation is faithful.

Two structural facts shape its grade. First, $K = \text{App}$ is free, so by [2, §2.3] the layer derivative is $T + T$ with the tag carrying the positional information, plug is injective, and contexts form a genuine coordinate system. Position is unforgeable, no nearness operator is needed, and the located stack of [1, §10] is implicit in the context $K([], S)$. Simulating the interaction structure is correspondingly cheap. Second, codes must be Church-encoded: λ can abstract over a hole but cannot treat the result as first-class transmissible data, so the representational half of the grade is expensive.

The second fact would be alarming were it not for confluence. Computing a digest of a de Bruijn form requires arithmetic, hence numerals and an interpreter, hence exactly the administrative machinery that cost CCS+rec its fidelity. It costs λ nothing but grade, because β is confluent: there are no choice points for administrative steps to relocate, so by Proposition 8.1 the second condition for kernelled laxity fails outright. λ is thus safe for a reason quite separate from its having a native plug — it would remain safe on the Gödel route — and that separation is what makes Proposition 8.1 worth stating with two clauses rather than one.

Remark 8.4 (A conjectural landing point). The order-forgetting erasure of [3, Rmk. 4.3] identifies two reduction paths when they differ by transposing independent steps. In the λ instance this should be Lévy’s permutation equivalence of reductions, and the corresponding fold of the universal cover the classical theory of developments. If so, the abstract erasure lattice acquires a well-developed concrete instance. We have not checked it; see Problem 2.

8.3 ρ : cheap codes, collapsed positions

ρ satisfies (I) twice over — by binding and by reflection, with @ making the representing term quotable in turn. Codes are primitive, so the representational half of the grade is cheap, and this is the design case for which C was built. But $K = |$ is associative-commutative, so by [2, §2.3] the layer derivative satisfies $\partial(\text{Bag } T) \cong \text{Bag } T$, plug is radically non-injective, positional structure has collapsed entirely, and the surface must be reified and matched by near. Simulating that — the bag, the nearness match, the located purse — is where ρ 's grade goes.

8.4 The profile

Probe	K	native plug	confluent	positional	representational	laxity
CCS+rec	AC	none	no	high	serialising	kernelled
λ	free	binding	yes	low	high (Church)	graded
ρ	AC	binding, reflection	no	high	low (primitive)	graded

Table 1: The three probes. All three Gödel-code their own contexts, so the “native plug” column records not what can be coded but what plugging costs; for CCS+rec the entry means *neither* absolutely nor relatively native, the latter by Remark 3.7 and hence conditional on Remark 8.3. “Positional” is the cost of simulating the interaction structure, “representational” the cost of getting a code back into head position. Only CCS+rec meets both conditions of Proposition 8.1, and only CCS+rec suffers kernelled laxity; λ escapes by confluence and ρ by having a native plug.

The finding is that the probes do not differ over whether internalisation *exists* — by Remark 3.6 it always does — nor even over whether a Gödel coding is available, since it always is. They differ over which laxity the internalisation suffers, and, among those suffering graded laxity, over where the grade goes. λ and ρ sit at opposite corners: a rigid K makes interaction cheap to simulate while forcing expensive codes; reflection makes codes cheap while an AC K forces an expensive simulation of position. No probe minimises both.

Conjecture 8.5 (The grade factors). *The grade of a faithful internalisation of C or H over a probe M factors as a product of a positional factor, determined by the equational theory of K , and a representational factor, determined by the route to self-coding. The two factors are independent: they are computed from the layer derivative and from the coding respectively.*

We regard Conjecture 8.5 as the main technical claim the three-probe analysis suggests and do not prove it here; see Problem 1.

9 The duality, as a hierarchy of internality

We can now state what the two monads are to each other.

C is resource-bounded forward evolution: a budget installed up front and drained by reduction, the consumed prefix of the temporal stack being the realised cost. H is the record required for reversal: a log produced by reduction and never drained, whose free instance is the universal cover and whose unique-parent property is backward determinism. The ledger law $\sigma(t) + \kappa(t) = \sigma_0$ binds them: one clock, two ledgers, read from opposite ends.

Under the algebra reading of Section 5 the duality is visible in what the algebras do. A C -algebra is a calculus that knows how to *pay*. H is a writer monad, so an H -algebra is

precisely an action of the event monoid — a calculus that knows how to *replay*. Pay and replay are the two erasure structures, and time reversal exchanges them.

One asymmetry is real and should not be smoothed away. H 's action only ever appends, which requires no decision; C 's must decide a surface match and a token match at every step, which requires a decidable guard supplied by the section. So the interpreter witnessing C 's internalisation carries a branch per step that H 's does not, and we should expect C 's grade to exceed H 's over any probe. That is a prediction the ledger law does not make and the algebra reading does.

Finally, the three probes turn the self-coding hypothesis from a side condition into the thing being measured, and the duality becomes a hierarchy rather than a single theorem.

CCS+rec: external.

Both C and H can be installed and neither faithfully internalised. The duality is real and cannot be stated inside the calculus; time is bookkeeping done from outside, which is what the reversible-CCS constructions do by hand.

λ : internal by encoding.

Both internalise faithfully, at a grade dominated by representation. The ledger law reads as a bounded reduction sequence carrying its own path — a legible, classical object, paid for in coding overhead.

ρ : internal primitively.

Both internalise faithfully at a grade dominated by position, and ρ can write its own metered reversible interpreter as a ρ term. The duality becomes a statement the theory makes about itself.

That ρ reaches the top of this hierarchy is not an accident of its expressive power but of the fusion noted in [1, §5]: @ serves as both the communication quotient and the authentication section, so the two distinct demands — code your canonical forms, code your contexts — are met by one operator. λ meets them by two unrelated mechanisms. CCS+rec meets neither.

10 Open problems

Problem 1 (The grade decomposition). Prove or refute Conjecture 8.5. The positional factor should be computable from the layer derivative of K — free giving injective plug and a coordinate system, AC giving $\partial(\text{Bag } T) \cong \text{Bag } T$ and collapse — and the representational factor from the route to self-coding. What must be checked is that they do not interact.

Problem 2 (The λ instance of order-forgetting). Is the Mazurkiewicz grade of [3, Rmk. 4.3], instantiated at λ , exactly Lévy's permutation equivalence of reductions? If so, is the corresponding intermediate cover the theory of developments?

Problem 3 (Does forgetting factor through internalising?). Is Forget the composite of a faithful internalisation with a quotient dropping the encoded apparatus? If so the two erasures are one map with a factorisation, and by Remark 7.3 the free and dissipative parts sit exactly on the seam.

Problem 4 (Discharging the uniformity hypothesis). Formulate the finitary uniformity condition of Remark 8.3 precisely, most likely as finite support in the sense of nominal sets, and verify it for every Ctx-morphism arising from an internally representable encoding. Concretely, the question is whether a CCS+rec encoding pre-allocating a fixed finite channel pool can simulate the name identification that $\mathcal{F}_\pi$ performs. This is the single step on which the CCS+rec column turns: settle it negatively and the ρ return leg of Remark 3.7 is blocked

and the column is unconditional; settle it positively and CCS+rec acquires a relatively native plug, its laxity becomes graded, and the probe order collapses above the Turing threshold.

Problem 5 (Is reflection necessary?). Reflection and erasure by binding are sufficient for faithful internalisation. Is either necessary? A negative answer would widen the class; a positive one would make “the theories over which the cost/history duality is internal” a structural characterisation rather than a list.

Problem 6 (A distributive law). Is there a distributive law relating C and H — metered history against history of metering? The history note carries the token apparatus unchanged on the first component [3, Def. 2.2], which is close to asserting one. This is where the duality would stop being an analogy between two constructions and become a theorem about their composite, and it is the natural sequel.

Problem 7 (Closure). Write down the argument, sketched in Remark 6.6, that $C(G)$ remains self-coding when G is, so that C restricts to the subcategory on which Definition 6.5 is available.

A correction carried forward

Independently of the above, Remark 2.12 of [2] corrects Table 1 of [1]: the λ row records the program-side surface as x , but read through the rigidity of App that entry names the *delivery site* — the locations inside the program continuation at which the environment’s contribution arrives — rather than the contact surface, which on both sides is the App location. This is why the environment-side entry is nominally degenerate rather than absent. Any revision of [1] should fold this in.

References

- [1] L. G. Meredith. *Continued Interactive GSLTs and the Cost Endofunctor: a construction one level up from cost-accounted Rholang*. F1R3FLY.io, 2026.
- [2] L. G. Meredith. *Probing Universality: replacing the Turing yardstick with a relative property for interactive GSLTs*. F1R3FLY.io, 2026.
- [3] L. G. Meredith. *History as an Endofunctor: reversibility, covering spaces, and the memory price of energy conservation in cost-accounted GSLTs*. F1R3FLY.io working note, 2026.
- [4] I. Phillips and I. Ulidowski. Reversing algebraic process calculi. *Journal of Logic and Algebraic Programming*, 73(1–2):70–96, 2007.
- [5] V. Danos and J. Krivine. Reversible communicating systems. In *CONCUR 2004*, LNCS 3170, pp. 292–307. Springer, 2004.
- [6] S. Mac Lane. *Categories for the Working Mathematician*, 2nd edition. Graduate Texts in Mathematics 5. Springer, 1998.
- [7] R. Landauer. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5(3):183–191, 1961.
- [8] C. H. Bennett. Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973.

- [9] J.-J. Lévy. *Réductions correctes et optimales dans le lambda-calcul*. Thèse de doctorat d'état, Université Paris VII, 1978.
- [10] C. McBride. The derivative of a regular type is its type of one-hole contexts. Unpublished manuscript, 2001.
- [11] D. Gorla. Towards a unified approach to encodability and separation results for process calculi. *Information and Computation*, 208(9):1031–1053, 2010.